

PERFORMANCE REPORT

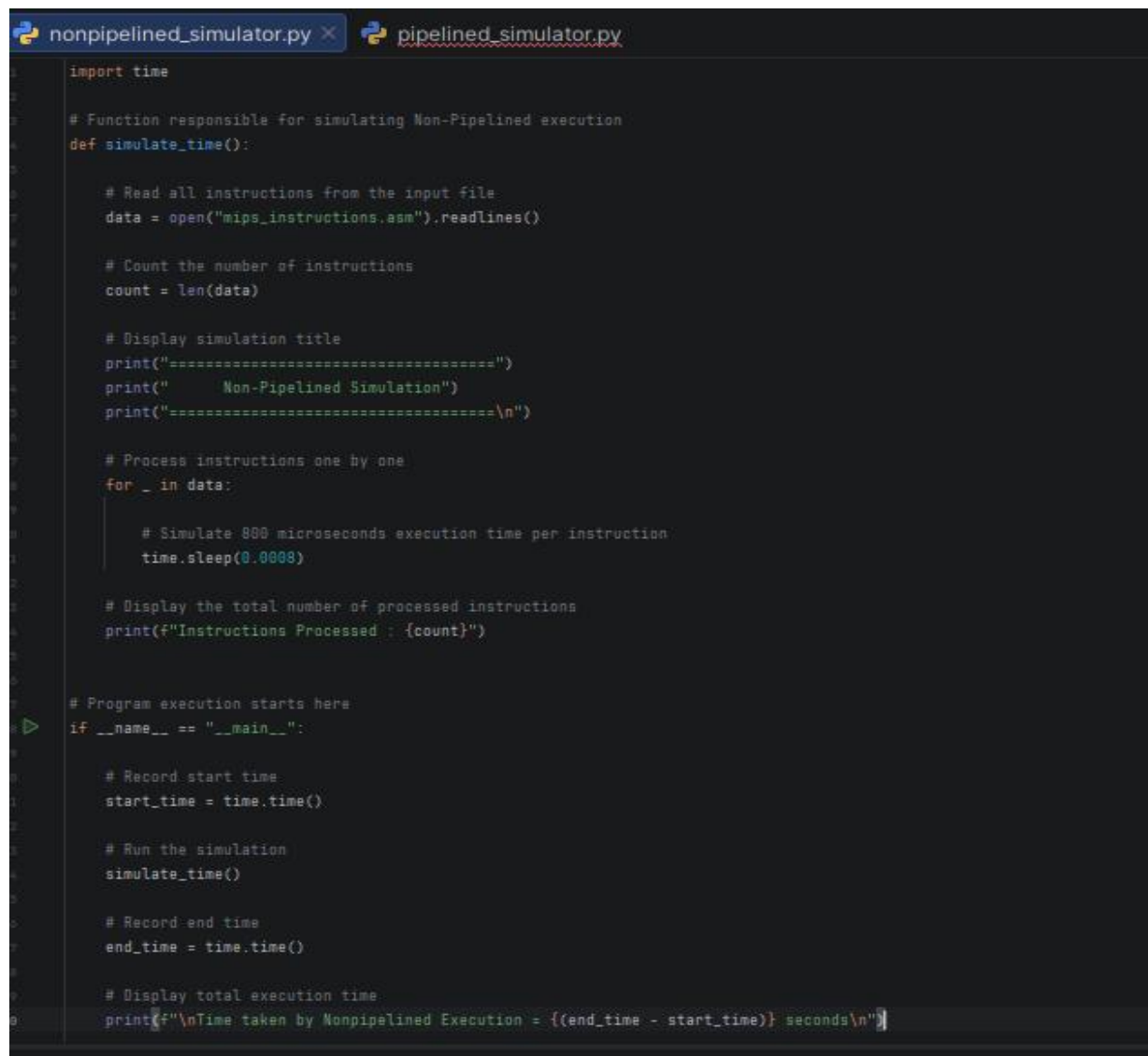
Between Pipelined And Nonpipelined Execution

Introduction

This Project was carried out to compare the execution time of two instruction processing methods: Pipelined and Nonpipelined execution. The purpose of the comparison is to observe how instruction overlap affects processor performance and how execution time changes as the number of instructions increases.

Process

About The Nonpipelined Code



```
import time

# Function responsible for simulating Non-Pipelined execution
def simulate_time():

    # Read all instructions from the input file
    data = open("mips_instructions.asm").readlines()

    # Count the number of instructions
    count = len(data)

    # Display simulation title
    print("=====")
    print("      Non-Pipelined Simulation")
    print("=====\\n")

    # Process instructions one by one
    for _ in data:

        # Simulate 800 microseconds execution time per instruction
        time.sleep(0.0008)

    # Display the total number of processed instructions
    print(f"Instructions Processed : {count}")

# Program execution starts here
if __name__ == "__main__":

    # Record start time
    start_time = time.time()

    # Run the simulation
    simulate_time()

    # Record end time
    end_time = time.time()

    # Display total execution time
    print(f"\\nTime taken by Nonpipelined Execution = {(end_time - start_time)} seconds\\n")
```

The Nonpipelined code simulates the simple way of instruction execution. The program reads all instructions from the `mips_instructions.asm` file and counts how many instructions exist.

After that, a loop is used to process each instruction separately. Inside the loop, a delay of 800 microseconds is added using `time.sleep(0.0008)` to represent the execution time of a single instruction.

At the end of the simulation, the program displays the number of processed instructions and measures the actual execution time using the start and end timestamps.

About The Pipelined Code

```
nonpipelined_simulator.py pipelined_simulator.py X
import time
import threading
import queue

# Function responsible for simulating Pipelined execution
def simulate_time():

    # Read all instructions from the input file
    data = open("mips_instructions.asm").readlines()

    # Count the number of instructions
    count = len(data)

    # Create 5 queues representing the pipeline stages
    stages = [queue.Queue() for _ in range(5)]

    # Display simulation title
    print("=====")
    print("      Pipelined Simulation")
    print("=====\\n")

    # Function executed by each pipeline stage
    def worker(i):

        # Keep the stage running
        while True:

            # Get instruction from current stage
            item = stages[i].get()

            # Stop thread when None is received
            if item is None:
                break

            # Simulate stage execution time
            time.sleep(0.0002)
```

```
nonpipelined_simulator.py pipelined_simulator.py X
def simulate_time():
    def worker(i):
        time.sleep(0.0002)

        # Pass instruction to the next stage
        if i < 4:
            stages[i + 1].put(item)

        # Mark current task as completed
        stages[i].task_done()

    # Store all pipeline threads
    threads = []

    # Create and start the 5 pipeline stages
    for i in range(5):
        t = threading.Thread(target=worker, args=(i,))
        t.start()
        threads.append(t)

    # Feed instructions into the first stage
    for instruction in data:
        stages[0].put(instruction)

        # Delay between stage starts
        time.sleep(0.0002)

    # Wait until all stages finish processing
    for stage in stages:
        stage.join()

    # Send termination signal to all stages
```

The Pipelined code simulates a five-stage instruction pipeline. The program first reads all instructions from the input file and then creates five stages that represent the pipeline structure. Instead of executing instructions sequentially, the code allows instructions to move through different stages at the same time.

A delay of 200 microseconds is used for each stage to simulate the timing behavior required by the project. Threads are used to represent the five stages and create the overlap between instructions.

After all instructions pass through the pipeline, the program calculates the total execution time and displays the final results.

Comparison

Nonpipelined Execution

In the Nonpipelined method, instructions are executed one after another. The processor must finish the current instruction before starting the next one. Because there is no overlap between instructions, the total execution time becomes larger as the workload increases.

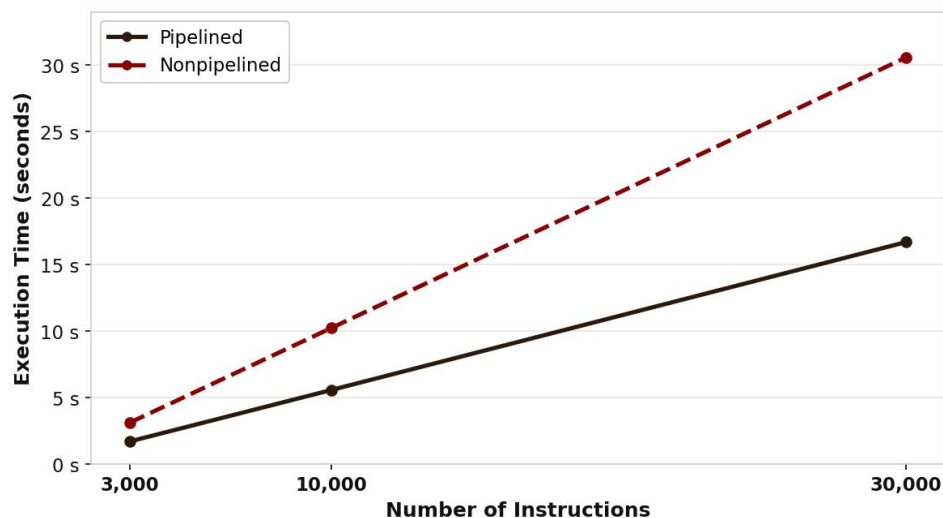
.2 Pipelined Execution

In the Pipelined method, instruction execution is divided into multiple stages. This allows several instructions to be processed simultaneously inside different stages of the pipeline.

Timing Results of Run

Number of Instructions	Pipelined (s)	Nonpipelined (s)
3,000	1.709 s	3.125 s
10,000	5.570 s	10.230 s
30,000	16.691 s	30.570 s

Graph



Pipelined execution is always faster

Comparing The Results

The results show that the Pipelined execution method have better performance than the Nonpipelined execution method.

These results indicate that the Pipelined method reduced execution time by allowing instruction overlap during processing.

The graph shows the execution time of both methods for different instruction counts.

The Nonpipelined method requires more time because instructions are processed sequentially, while the Pipelined method allows multiple instructions to be Work inside different stages at the same time.